Map editor – analysis (Pilotwings 64, USA)

As of 28 October 2026. Analysis for the map editor of Hawk's Hangar (Map and Objects tabs). Nothing of this is built yet. Sources: the decompilation (https://github.com/gcsmith/Pilotwings64Decomp, MIT license; names of functions and variables are taken from it), its `docs/pilotwings64_filesystem.md`, and the Pilotwings 64 viewer of noclip.website (https://github.com/magcius/noclip.website, `src/Pilotwings64/Scenes.ts`, MIT license), which already reads and draws islands and mission objects. Every format below was checked with a prototype reader against all files of the retail ROM.

1. How an island is built

File	Count	What it holds
UVTR (1 file, 10 COMM chunks)	10 terrains	a grid of cells; each used cell places one contour (UVCT) at its centre
UVCT	101	a contour = one piece of terrain: vertices, triangles, materials (texture + triangle range), collision flags, and the static models standing on it (buildings, trees …) with their placement matrices
UVMD	363	models (LODs, parts with part matrices, materials); used for the static models of the contours, the island's toys, the vehicles and pilots, and all mission objects
UVTX	463	textures (the Textures tab already decodes them)
UVLV (1 file, 136 chunks)	levels	lists of terrains, models, contours, textures … that one level loads; islands are levels 1, 3, 5, 10
UPWL (user files 0–3)	4	island data: landing pad places (LPAD), wind objects (WOBJ), toys (TOYS), terrain switch points (TPTS), audio points (APTS), environment sounds (ESND), the Birdman's bonus star (BNUS)
UPWT (user files 5–65)	61	missions: COMM (counts, scoring …) and one chunk per object type

1.1 Terrains (UVTR COMM)

Header: bounding box (min x, y, z, max x, y, z as f32, 0x00–0x17), grid width and height (u8, 0x18/0x19), cell size x/y (f32, 0x1A/0x1E), the cell's bounding radius (f32, 0x22). Then per cell one flag byte; a used cell has 0x43 more bytes: an identity 4×3 matrix (unused), the cell centre (f32 x, y, z; z always 0), 1.0, a rotation byte (always 0) and the contour number (u16).

Terrain	Island	Grid	Cell	Size (units)	Used cells	Note
0	Holiday	2 × 2	600	1200 × 1200	4	
1	Crescent	8 × 8	512	4096 × 4096	45	
2	Crescent	8 × 8	512		2	second terrain of the island (switch points)
3	Little States	8 × 5	1000	8000 × 5000	38	
4–6	Little States	8 × 5	1000		3 / 11 / 5	not in the island's level list
7	Ever-Frost	2 × 5	1000	2000 × 5000	10	
8	Ever-Frost	2 × 5	1000		1	second terrain of the island
9	Sky Diving free fall	1 × 1	6000		1	

Island level lists (UVLV): Holiday 1 terrain / 38 models / 4 contours / 70 textures; Crescent terrains 1 and 2 / 54 / 47 / 107; Little States terrain 3 / 103 / 38 / 147; Ever-Frost terrains 7 and 8 / 40 / 11 / 95.

Coordinates: x east, y north, z up (game units; 1 unit = 0.7 game meters).

1.2 Contours (UVCT COMM)

u16 vertex count, triangle count, model count, material count; vertices of 0x10 bytes (s16 x, y, z, pad, s16 s, t texture coordinates, u8 r, g, b, a vertex colour); triangles of 8 bytes (3 × u16 vertex, u16 collision flags); per placed model: a count of N64 fixed-point matrices (64 bytes each, the first is the placement), s16 model id (−1 = none), f32 x, y, z (repeats the matrix's translation), 4 bytes; per material: u16 render mode, u16 texture number, u16 vertex count, u16 triangle count, u16 command count, the commands (u16, plus one byte for a vertex load), u16 first triangle, u16 solid, u16 sub-grid flags (4 × 4 cells, used by collision), more bytes (0x18 in all); at the end a bounding sphere (f32 x, y, z, radius) and padding. All 101 contours: 29,768 vertices, 20,846 triangles, 1,364 placed models of 183 different models.

1.3 Models (UVMD COMM)

u16 vertex count, u8 LOD count, u8 part count, u8 count of 0x24-byte records, u8 transparency; vertices as in UVCT; per LOD: u8 part count, u8 billboard, per part u8 material count, u8, u8 attachment level, per material the same header and commands as UVCT (commands: 0x4000 |

three 4-bit indices into the 16-vertex buffer = a triangle; otherwise a vertex load with a count byte), f32 LOD distance; then the part matrices (4 × 4 f32), the 0x24-byte records and the inverse scale (f32). All 363 models parse. The four island maps (models 0x00, 0x28, 0x51, 0xA5) are whole islands at 1/10 size for the game's map view (2,405 / 1,420 / 5,185 / 3,637 triangles).

1.4 Prototype pictures

A prototype (Python) drew all four islands from their contours and the placed models, coloured with the average colour of each texture times the vertex colour, from above and in perspective, and the objects of missions on top (rings with their child links, start, runway, landing pad, thermals). Islands, buildings, and the objects of the missions line up with the game (e.g. GC Class A: start on the runway, the 15 rings around Crescent Island).

2. Mission objects (UPWT chunks)

Every chunk is `count × record size` (count in COMM 0x41C–0x42B); the loader copies the chunk with its file size into a block allocated from the count, so both must match (the editor writes both). Retail use of all 61 missions: rings 122, start 61, thermals 60, targets 43, landing pads 18, hover pads 16, Meca Hawk domains 14, Cannonball targets 12, local winds 11, runways 9, photo entries 6, balloons 5, hopper goals 3, ball targets 2; SDFM and OBSV never.

2.1 Meca Hawk (FALC, 0xAC bytes; GC B3 6 records, GC P3 8)

Each record is a domain, a sphere (centre 0x00, radius 0x0C). There is one Meca Hawk: the one record with byte 0x10 = 1 is its start and its starting domain and gives all its behaviour values; the other records only use centre and radius. While moving, the hawk leaves its domain when its x/y distance from the centre exceeds the radius; it then takes the first domain (in record order) whose sphere contains it, else it turns back to its domain's centre. The hawk walks at ground + 27.63 (in water between +1 and −9), so a domain's centre height must let the sphere contain that height.

Offset	Meaning	Retail (B3 / P3)
0x00 Vec3F	domain centre; for the start record also the start x/y (z is set to the ground)	
0x0C f32	domain radius	400 / 300
0x10 u8	1 = the hawk starts here, behaviour values from this record	one record per mission
0x11/0x12/0x13 u8	% chance of walking, running (swimming in water), diving (water only) when a state ends; the rest is standing	40/25/0, 30/25/10
0x14 u8	the standing share (100 − the three), not read	
0x18–0x4C f32 pairs	durations min/max (s): walk, run, walk in water, swim, hit stagger, dive (original bug: range 0x44 – 0x38), stand (also after a throw)	
0x50–0x7C f32 pairs	speeds min/max (units/s): walk, run, walk in water, swim, hit stagger, dive	
0x80/0x84	speed of the thrown rock min/max	230–260 / 270–300
0x88/0x8C	lifetime of the rock min/max (s)	3.1–3.3 / 3–3.1
0x90/0x94	the player's distance for throwing: min, max	250–700 / 250–800
0x98	pause between throws (s)	2.8 / 2.6
0x9C/0xA0	copied, never read	
0xA4 f32	points per missile hit (count only if the hawk is destroyed)	6
0xA8 f32	missile hits needed (0 = 5)	5

A throw needs: standing, not under water, the player between min and max distance, the player looking at the hawk (about ±15°), the pause over. Limits: at most 12 records (more: no hawk at all); exactly one start record (none: no hawk but the mission still waits for it; two: a second set of models and a read behind sFalcos[1] in the map). Models: hawk 0xD3, in water 0xFD/0xFE, rock 0xFF (in water 0x100), at most 16 rocks.

2.2 Missile targets (TARG, 0x20 bytes; GC A2 3, B2 10, P2 30)

Offset	Meaning
0x00 Vec3F	position (also radar and map)
0x0C/0x10/0x14 f32	heading, then two more rotations in degrees (retail only the heading; applied z, x, y)
0x18 u8	type: 0 ball (model 0xF9, spins 0.5 turns/s, can also be flown into), 1 disc (0xF8), 2 billboard (0xF7); others: no model, still counted
0x19 u8	points per target (retail 1 / 3 / 10)

Original quirk: every ball takes the orientation of the first ball. At most 40 (raisable to 64), more: none load. Each uses one radar waypoint (70) and one map object.

2.3 Local winds (LWIN, 0x54; 11 records in 7 missions)

Offset	Meaning
0x00 Vec3F	sphere centre (shape 0) or cylinder end A (shape 1)
0x0C Vec3F	cylinder end B (shape 1)
0x18 Vec3F	box centre (shape 2)
0x24 Vec3F	box rotation (degrees; heading, pitch, roll)
0x30 Vec3F	box size (±size/2 in the box's axes; local y the long axis); must not be 0 (division)
0x3C s32	area number in which its sound plays (10 = all); not a limit of the force
0x40 Vec3F	wind vector (world), scaled by the falloff and added to the global wind; sound volume from its length
0x4C f32	radius for shapes 0/1 (for a box the game sets it to the size's length)
0x50 u8	shape: 0 sphere, 1 cylinder, 2 rotated box (retail all 2); others stop loading the remaining winds
0x51 u8	falloff over the normalised distance: 0 constant, 1 linear, 2 quadratic, 3 flat to 0.8 then linear (>3 undefined)

Models 0xD5 (sphere/cylinder) and 0xD6 (box), scaled. Not on map or radar. At most 6

(raisable to 16).

2.4 Thermals (THER, 0x28; Hang Glider)

0x00 base centre (bottom of the cylinder), 0x0C radius (2D; model and radar circle), 0x10 height (must not be 0), 0x14 area for the sound, 0x18 bottom ramp height (lift × z / ramp below it), 0x1C top fade height, 0x20 swirl (retail 0), 0x24 lift at the centre (× √((r − d) / r); retail 30–50). Model 0x101 (map 0xC7), orange radar circle within 800 units. At most 16 (raisable to 20, the radar's list). HG P1: thermals stop after 4:10 (code).

2.5 Start (TPAD, 0x30; one per mission)

0x00 position, 0x0C rotation (degrees: heading, pitch, roll), 0x1C start velocity in the vehicle's frame (y forward, z up), 0x28 0 = on the ground (snapped, but only within ±0.5 units of z: the start must lie on the terrain), else in the air, 0x2C start fuel 0–1 (RB, GC). Per vehicle: HG/RB/SD/BM use the velocity, Cannonball and JH ignore it; Cannonball and Birdman use the position as it is; Sky Diving starts 1000 units below z. Only the first record is used (at most 3 loaded). Map model 0xC8 (not for Birdman and Cannonball). Birdman bonus star: the starts of the Birdman missions with test 0 are kept per class and copied over the start when a star swaps to the Birdman.

2.6 Ball target (BTGT, 0x1C), hopper goal (HOPD, 0x20), Cannonball target (CNTG, 0x1C), OBSV (0x10)

- BTGT: 0x00 base, 0x0C area (only shown and counted there), 0x10 radius, 0x14 height, 0x18 unused, 0x19 points. Models 0xF3 / 0xD2 (scored). A ball inside ends the mission. At most 5.
- HOPD: 0x00 area, 0x04 position, 0x10 1 = ends the mission, 0x14 radius (2D), 0x18 model height. Models 0x109 / 0xD1. At most 5 (several work only with goal-objects-v1). Radar goals: HOPD + BTGT + used pads + runways (the module keeps all).
- CNTG: 0x00 position, 0x0C rotation, 0x18 size 0/1/2 (models 0x106–0x108, bullseye 35 / 42.5 / 50 units up). One per mission.
- OBSV (never used): a fixed camera point and a trigger radius; while the vehicle is inside, the camera looks at it from there. No limit, no model.

Chunks are padded to 8 bytes in the files (the allocator rounds up to 8 as well).

2.7 Rings (RNGS, 0x84)

Offset	Meaning
0x00 Vec3F	position (anchor of a moving ring)
0x0C Vec3F	rotation in degrees (heading, pitch, roll; order z, x, y); the ring's local +y is the direction to fly through
0x18 s32	area: shown and active only while the player is in it (0 island, 1 the cave of RB P1)
0x1C u8	not read; retail equals the score type (+0x72), the editor keeps it so
0x1D u8, 0x20 s32[5]	child rings (⩽ 5, indices < count; not checked)
0x34 u8, 0x38 s32[5]	timed child rings (⩽ 5)
0x4C u8	points for passing the ring while its timer runs (instead of the normal points)
0x4D u8	1 = can be a timed ring (the decomp's name "untimed" is wrong); 0: ignored as a timed child
0x50 f32	timer in seconds (the dial is full at 20 s)
0x54 u8	size 0–4: radius 7.5 / 10 / 12.5 / 17.5 / 25 units
0x55 u8	active at the start
0x58 f32, 0x5C f32, 0x60 char	movement: speed (rad/s, applied × 0.5), radius, axis 'x' / 'y' / 'z' ('n' none)
0x64 / 0x68 / 0x6C f32, 0x70 char	spin before a timer, while it runs, after it ran out (rad/s), spin axis
0x71 u8	0 one way only (from local −y to +y; wrong way buzzes), 1 both ways; must be 0 or 1 (original range check tests the wrong field: 2+ gives a wrong model and an unpassable ring)
0x72 u8	score type: 0 normal, 1 bonus, 2 invisible check ring, 3 goal ring (> 3 becomes 3)
0x74 char[16]	name, debug only, at most 14 characters

- Model (rings.c `sRingModelIdLookup`): active type 0/1: 0xD9 + 4 × size + 2 × (has child rings: +0x1D ≠ 0, blue / white instead of yellow / red) + type; goal ring 0xF1; check ring none; inactive: lock 0xEF (type 0), 0xF0 (types 1, 3). (Corrected 29 October 2026: earlier this read "2 × (+0x71)".)
- Model parts (rings.c 803234A4): the ring models carry two dials, the game hides part 4 when +0x71 = 0, otherwise part 1; the clock hands (parts 2 / 3 for +0x71 = 0, 5 / 6

otherwise) are hidden until the ring becomes a timed ring. Then hand 2 / 5 stays at zero and hand 3 / 6 turns about its own y axis by 18° per second left (`uvMat4RotateAxis` on the part's matrix). The editor shows the hands of every ring that can become a timed ring (+0x4D ≠ 0, timer > 0) with the second hand at the full timer.

- Logic: only active rings can be passed (crossing the ring's plane within its radius). Passing activates its children; they are alternatives: passing one removes its unpassed siblings (cleared, no points). The open children are one shared list, so two branching rings open at the same time interfere. Passing starts the timers of its timed children (normally rings already active); in time they give their own points, afterwards their normal points. The "all rings" condition needs every ring cleared, skipped branches included.
- Points: COMM+0x3E4 s16[4] per score type (Mission_Scoring.md 2.7). At most 30 (raisable to 64).

2.8 Floating Pads (HPAD, 0x40; RB A2 6, RB P3 10)

0x00 position, 0x0C rotation, 0x18 area, 0x1C 0 large (0xFC) / 1 standard (0xFB), 0x1D points, 0x20 fuel added (fraction of a tank), 0x24 u8 + 0x28 s32[5] the pads activated after landing here, 0x3C active at the start (inactive: model 0xFA). Retail: simple chains. At most 20 (raisable to 40).

2.9 Balloons and balls (BALS, 0x68)

0x00 centre, 0x18 area, 0x1C sideways speed of the parts when it splits, 0x20 colour 0 orange / 1 green / 2 blue, 0x21 splits into 5 when popped, 0x24 pop speed (relative speed; 500 = never), 0x28 bounce factor (0 = does not move), 0x2C mass, 0x30 scale (radius 5.324 × scale), 0x34 points, 0x3C drag, 0x40 gravity (0 = 9.8), 0x44–0x67 the same values for the five parts. No type flag: a balloon stands still and pops easily; the ball for the ball target never pops and moves (RB B3, P2). At most 20 including the parts (raisable to 40).

2.10 Landing pad (LPAD, 0x30) and runway (LSTP, 0x24)

- LPAD: 0x00 position, 0x2C type 0/1/2 (models 0x102–0x104: target circle radius 10 / 15 / 25; ⩾ 3 reads garbage). The pad snaps to the nearest free island pad place (UPWL LPAD) within 100 units and takes its position and angle; with none in range it is dead (no model, radar, landing). The island pad place's angle (UPWL LPAD +0x0C) is in **radians**: pads.c hands it to `func_80313640` without converting (the Cannonball target, by contrast, converts degrees × 0.01745). Island pad places: Holiday 1, Crescent 6, Little States 4, Ever-Frost 5 (at most 14). Sky Diving needs exactly one pad and judges "LAND OK" within 25 units.
- LSTP: 0x00 and 0x0C the two ends of the centre line (not corners), 0x1C landing is checked only when not 0, 0x20 width. At most 2.

3. Island data (UPWL)

Island	LPAD places	WOBJ	TOYS	TPTS	BNUS
Holiday	1	0	3	0	1
Crescent	6	6	0	3	1
Little States	4	13	13	4	1
Ever-Frost	5	0	0	1	1

Limits (level.c): WOBJ 16, LPAD 14, TOYS 16, TPTS 16, APTS 20, BNUS 2. These belong to the island, so a change applies to every mission flown there. The moving toys (boats, gliders, planes, ski lift) are placed by code per island, not by data.

3.1 Records (researched 29 October 2026)

- **LPAD** 0x18: position, f32 angle in radians, isUsed, type.
- **TPTS** 0x34 (area switch / warp point, task.c 396–476): u8 mode at 0 (0 on entering, 1 only when crossing from the back to the front), position at 4, angle in degrees at 0x10, radius at 0x14, s32 front area at 0x18, s32 back area at 0x20, offset at 0x28. Inside the sphere the side of the plane facing angle + 90° chooses the area; when the area changes, the offset is added to the player's position. Area −1 = only a warp (Little States' hangars and office tower).
- **APTS** 0x20 (sound zone, env_sound.c 444–519): position, angle 0x0C, radius 0x10, s32 type 0x14, values 0x18 / 0x1C. Type 1: a ray upwards by the radius; flying out from under cover plays a whoosh. Type 0: the volume of the surroundings fades between the two values on both sides (cave mouths).
- **BNUS** 0x1C: position, heading / pitch / roll in degrees, s32 area at 0x18 (Ever-Frost's star is in the ice cave).
- **TOYS** 0x10: only the position of an animated terrain model (merry-go-round, ferris wheel, water wheel, Mount Rushmore, pump jacks); toys.c moves its parts within 100 units.
- **ESND** 0x78 (sound emitter, env_sound.c 90–123, audio_emitter.c): matrix (position = translation at 0x30), line points A 0x40 / B 0x4C, u8 sound 0x58 (index into `sEnvSoundModelIdLookup`), pitch 0x5C, volume 0x60, priority 0x64, near 0x68 and far 0x6C radius (squared on load; the volume falls off linearly in the squared distance), u8 zone 0x70 (10 = everywhere; only Crescent switches by it), attributes 0x74 (0x2 = line from A to B, 0x8 = 3D distance). Holiday 43 (38 lines), Crescent 166, Little States 192, Ever-Frost 210.

3.2 Sub-areas

The area number maps to a terrain per island (task.c 713–755): Crescent 0 → terrain 1, 1 → terrain 2 (cave and volcano); Ever-Frost 0 → 7, 1 → 8 (ice cave). Little States' terrains 4–6 are the award ceremony (chosen by vehicle) and cannot be flown to; terrain 9 is the Sky Diving backdrop. The Map tab offers the two caves as entries of their own in the island list.

3.3 Objects placed by the game's code (`IslandExtra`)

Circular movers: position = centre + rotate_z(angle)·(r, 0, 0), speed in degrees per second, the model faces along the circle. Holiday: speedboat 0x03 (−600,−600) r 300 −4°/s, sailboat 0x01 (700,−500) r 300 +2°/s, three hang gliders, night switch sphere (−82,250,51) r 6 (environment.c). Crescent: speedboat, yacht 0x29, three gliders, whale 0xD7 (750,100) r 75, fountain 0x26. Little States: four gliders, ferry 0x52 (1150,−2150) r 200, Missi 0x58, space shuttle 0x56 (2870,−2230), three fuel station boxes ±10 × ±10 × ±2 (code_66F70.c; Gyrocopter class B, Rocket Belt class A), two planes on spline paths (SPTH user files 109 / 110, not shown yet). Ever-Frost: three gliders, whale pod (five whales around (1156.66,1770.82)), oil spray 0xA6, two boxes without snow (snow.c), ski lift on a spline (not shown yet). The gliders' models depend on the chosen pilot (the player's own pilot is skipped); the editor shows those of a game with Lark. Fire effects (sprites) are not shown. The noclip viewer has two wrong values: Holiday's sailboat turns 2°/s (not 10), Ever-Frost's second glider has radius 100 (not 168).

Spline paths (SPTH user files; spath.c): chunks SCPX/Y/Z/H/P/R, each u32 count, count × (f32 time 0–100, f32 value), 4 zero bytes; first and last key equal. Cubic Hermite segments. Planes (planes.c, user files 109 / 110, models 0x27 / 0x1B, a loop in 120 s, lowest z 42.43 / 33.76): H/P/R hold dx/dt, dy/dt, dz/dt, heading = atan2(−vx, vy). Ski lift (ski_lift.c, user file 4, 20 chairs 0xA7 at t = 5 × i, +0.03 per frame): tangents = direction to the neighbouring key × |Δt| × 0.01 × 512, heading from the SCPH keys in degrees (−45° up, +135° down). Fires (fire_effects.c, oil_plant.c): 51 + 1 flames, a quad of width × height standing on its base that turns to the camera about z (fx.c), 16 pictures (UVTX 0x45–0x54, IA16 16 × 32, 20 per second), colour = T + (1 − T) × prim 0x915600; Crescent 8 on the island and 37 in the cave, Little States 4, Ever-Frost 2 + the oil plant (only within 500 units). Model 0x03F is in Crescent's level list but placed nowhere and used by no code (the decompilation: "Bridge?? flat with ramp, not sure if used").

3.5 Colour combiner, paired textures, decals

Every texture's display list sets the RDP colour combiner (G_SETCOMBINE, two cycles of (a − b) × c + d for colour and alpha), often primitive / environment colours, and up to two tiles: texel 0 is the tile G_TEXTURE names, texel 1 the next one. Common cases: lerp(T0, T1, LOD fraction) × shade (mipmaps); T0 × T1 × shade with tile 1 on the same image at another scale (detail); a texture that loads a **paired texture's** image (second G_SETTIMG; trailer u16 at +9 after the display list) into tile 0, e.g. the Grand Canyon: colour image 0xAC × own grey detail image 0xAD; T + (1 − T) × prim (fire); alpha from the shade only (terrain), so dark texels of I-format textures must not be cut out. Material mode bit 8 is the decal mode (RDP modes 8–15): markings, windows and signs drawn on a surface at the same depth (53 models, 28 contours).

3.6 Pause menu map (map3d.c)

The map model is drawn at identity (map units = world × 0.1); markers sit at the objects' real positions × 0.1 at their model's own size, i.e. ten times their size in world units. Rings and Floating Pads: 0xCA yellow when active, 0xCC blue when inactive (invisible check rings none; passed ones disappear while flying); balls/balloons and missile targets 0xCA; landing pads and runways (midpoint) the arrow 0xC9 × 0.3; the start the arrow 0xC8 × 0.3 turned by its

heading; thermals the cylinder 0xC7 (x, y = 2 × scale / 1000 × 4, z = height / 1000 × 4); hopper goal 0xD1 and ball target 0xD2 at their size; Meca Hawk 0xD3 (only the record that starts it); Cannonball targets 0xD0. With the Cannonball only its targets, with the Birdman nothing. Rings and Floating Pads only of the player's current area. The player is the arrow 0xCE; labels are 2D sprites.

3.4 Texture palettes (UVTP)

The game's only UVTP file holds seven palettes as COMM chunks; each is a u16 count and pairs of u16 (original texture, replacement). envLoadTerrainPal (environment.c) loads one per environment: 6 → 0, 11 → 1, 17 → 2, 18 / 19 → 5, 20 → 4, 21 → 3; the level's textures are swapped while loading. Ever-Frost's clear day (environment 18) uses palette 5.

4. Proposed plan (to be decided by the user)

1. Core readers for terrains, contours, models, island data and every mission object type (typed records with the meanings above), tested against the ROM; a software renderer in the Core (top view and perspective, textures from the existing texture decoder); the Map tab read-only: island, mission, objects, selection and properties.
2. Moving and turning objects (view and number fields, "on the ground"), undo, saving into the mission file, import.
3. Adding and removing objects, the limits ("raise limit" slider), the warnings (memory, display objects, radar).
4. The special editors: ring links and timers, Floating Pad chains, Meca Hawk domains and behaviour, winds, thermals, balloons and balls.
5. Island data (landing pad places, wind objects, bonus star) if wanted.
6. Sample photo preview, display list scan, space per island. Then the user's tests of Steps 44–46.

5. How the files are organised (by type, not by island or mission)

- The file system holds 1272 files **sorted by type** (terrains, levels, contours, models, textures …, then the user files); a file's number is its place among the files of its type. Nothing is grouped by island.
- What belongs to an island is a **list** in its level (UVLV levels 1, 3, 5, 10): its terrains, models, contours and textures (Holiday 38 / 4 / 70, Crescent 54 / 47 / 107, Little States 103 / 38 / 147, Ever-Frost 40 / 11 / 95). Every contour belongs to exactly one island; models and textures mostly too (21 / 29 / 78 / 33 models and 16 / 17 / 58 / 48 textures are used by one island only; 1 model and 12 textures by all four).
- The 168 models in no island list (vehicles, pilots, the mission objects such as rings and balloons, menus) are loaded by the game's other levels (vehicle and pilot levels, 0x1A, 0x1B, 0x0C, 0x0D, 0x2E in levelLoad), the same on every island.
- Missions: one file per mission (UPWT) with **all its objects**; its island is COMM byte 3 (and the class table).
- Island data: one file per island (UPWL).

- The map model of the pause menu (0x00, 0x28, 0x51, 0xA5) has its vertices in world units (its inverse scale 10 makes it 1/10 in the game's map view), so drawn at scale 1 it lies exactly on the island (extents checked).

6. Decisions of the user (28 October 2026)

- Display: mainly the editor's own drawing (Core renderer); in addition an option to draw with the graphics card (WPF 3D). The pause menu's map model can be laid transparently over the island.
- Map tab: shows the islands, also the coarse map models of the pause menu, and a list of the objects with their properties. The user will send pictures for the GUI first; **no code before those pictures**.
- Objects tab: shows the game's models (the islands and all single objects used in the game); replacing models (the original must always be restorable) and adding new models for more islands and new objects.
- The first step already includes moving objects (not only showing them).
- Island data (landing pad places, wind objects, bonus star): editable later, as a step of its own.

7. GUI of the Map tab (user's sketches, 28 October 2026)

- Tabs: **Model** = show, replace (always restorable) and add models (islands and single objects); the analysis colouring of detail and critical places belongs there. **Objects** = give models behaviours and make finished objects from them, which the map then uses. **Map** = the map editor below.
- Layout: left the object list (the mission choice may sit above it) and below it the details of the chosen object; top right "Load" (dropdowns island and mission) and "View" (normal, untextured white, wireframe); the map in the middle with a move gizmo (red/green/blue axis arrows and a yellow cube); a bar at the bottom with Mouse, Camera, Object Transform.
- Mission choice: one mission, or all missions of one vehicle on the island (not all vehicles together). Besides the mission objects the island's decoration objects are shown and must be editable.
- **Mouse:** a mode dropdown: (1) navigate only (left pan, right rotate, wheel zoom); (2) select and move (left select, right pan, middle rotate, wheel zoom); (3) user defined: only then the dropdowns Left / Middle / Right click and Wheel are active.
- **Camera:** Panning or Orbit (orbit: the camera looks at the chosen object, right button orbits around it); a View dropdown: island, map model, both overlaid (then a slider for the map model's transparency). Panning: a 2D field (drag with the left button, also beyond the box: the box is only where the drag starts), a vertical 1D field (forward/back), a horizontal 1D field (zoom), X/Y/Z number fields with up/down arrows and a 1D drag field each. Rotation: a 2D field and X/Y/Z rows the same way.
- **Object Transform:** Movement (2D field, vertical 1D field, X/Y/Z rows) and Rotation (2D field, X/Y/Z rows) for the chosen object, the same controls.

- Further decisions (28 October 2026, second round): the yellow cube is the chosen object's bounding box; dragging it moves the object freely; in rotation mode the gizmo shows rings for rotating. The 2D fields work relative to the camera (object: drag up = away from the viewer, the vertical bar = height; rotation: left/right heading, up/down pitch, roll only in its row; camera in orbit mode: the rotation field orbits the object). Positions in game meters, angles in degrees, X east, Y north, Z up. User-defined mouse actions: nothing, select, pan view, rotate view, move object, rotate object; wheel: zoom, forward/back, object height; stored per user (settings.json). Several objects at once: later. Undo/redo (Ctrl+Z / Ctrl+Y): yes. A mode without mission objects (decoration only). With all missions of a vehicle shown the objects stay editable. The View section gets a check box to show the tiles (vertical lines at the tiles' corners). Decoration objects are edited in a step after the first. The island's moving toys: later. Behaviors tab: view the code of objects, add new code or copy and edit existing. Switching to drawing with the graphics card: later, in "View".
- Third round (28 October 2026): buttons above the object list to add and remove objects; dropdowns at the top of the details to switch the object; the camera bar can switch to the axis views (top and the four sides); Object Transform gets a button to place the object on the ground below it; landing pads as proposed (more settings later); a scale slider in the details; a speed slider for the camera tools and one for the object tools; the gizmo switches between move and rotate with two toggle buttons in the Object Transform header and the keys W / E (changed to Q / E in the rework); Step 51 comes in two ZIPs (51a display, camera, mouse, selection; 51b moving, rotating, undo, saving); while dragging the view is drawn at half resolution.

8. Built in Step 51a: reading, drawing and choosing (Map tab, part 1)

- **Core readers** (`PW64Editor.Core.Map`): `TerrainMap` (UVTR), `Contour` (UVCT, with the placed decoration models and their placement matrices), `GameModel` (UVMD, detail levels, parts with their tree, billboards, inverse scale), `LevelList` (UVLV), `IslandFile` (UPWL: pad places, wind objects, toys, bonus stars; read only), `EnvironmentModels` (UVEN: sky and sea models), `MapIsland` (level, map model, environment and island file of the four islands), `MapLibrary` (reads the project's current files and keeps what it read), `ModelNames` (the decompilation's model enum in plain words). Mission objects: `MapObjectSchema` (record size, placement, named fields with units per kind; the meanings of section 2) and `MapObject` (one record, position, angles, field values as text in game meters, degrees …).
- **Drawing** (`PW64Editor.Core.Map.Rendering`): `RenderMesh` (contour or model as triangles with texture coordinates, vertex colours and normals; cut-out textures drawn opaque with an alpha test, partly transparent ones blended), `MapScene` / `MapSceneBuilder` (terrain, decoration, island objects, sea, map model and the mission objects with the models of section 2; lines for runways, check rings, domains, winds, ring and pad links), `MapRenderer` (software: perspective-correct textures, depth buffer, near-plane clipping, wrap / mirror / clamp, views textured / white / wireframe, the map model laid over with a chosen opacity, tile lines, bounding box and move/rotate handles of the

chosen object, a pick number per pixel; drawn in horizontal bands on all cores), `MapCamera` (perspective and the parallel axis views top / north / south / west / east) and `MapCameraController` (pan, turn, orbit around the chosen object, zoom to the view's centre, forward/back at the same height, field of view), `MapMouseSettings` (the three mouse modes and the user-defined buttons, kept per user in settings.json). `MapScene.GroundHeight` gives the terrain height at a point (for "on the ground").

- **Map tab:** object list (mission objects by type, island objects, decoration by tile) with the details of the chosen object (all fields with names and units, a note on the game's rules), the detail dropdowns (group, object), View (textured, white, wireframe, tiles), Load (island; decoration only, all missions of a vehicle, one mission), the map (click to choose, mouse modes, the point under the mouse in game meters), the bar at the bottom (mouse, camera with panning/orbit, island / map model / both with the transparency, axis views, speed, the 2D/1D drag fields and the X/Y/Z rows). The Object Transform tools and the Add/Remove buttons show the values but are not active yet (Step 51b). The tab reads the files again when it is shown after files of the project changed.
- **Checked:** harness (readers against all files: 10 terrains, 101 contours with 29,768 vertices and 1,364 placed models, 363 models, 136 levels, the island pads 1 / 6 / 4 / 5, all mission objects by kind; the ground starts of the Gyrocopter lie on the computed ground within 1.5 units; every island drawn in all views; every start found again by its pick number in a top view; camera controller). Pictures of all four islands looked at (textured, white, wireframe, top view, map model, map model laid over, close-ups of start, ring, Floating Pads, target, balloon, Cannonball target). Not checked: the WPF tab itself (no Windows here).
- Rework of Step 51a (29 October 2026, user's test): a slider reported its value while InitializeComponent still built the tab (NullReferenceException); the tab now ignores control events until it is set up. The handles switch with **Q** / **E** (move / rotate). **W A S D** move the view while the map has the keyboard focus (after a click into it): axis views up/left/down/right in the picture (from above: north, west, south, east), perspective forward/back and sideways at the same height (the direction stays; it changes with the mouse), orbit mode around the chosen object (`MapCameraController.MoveByKeys`); not with Ctrl held (Ctrl+S).
- Second rework of Step 51a (29 October 2026, user's test list):
  - **Landing pads:** the island pad places' angle is radians (see 2.10), so pads now lie parallel to their runway; pads and runways get a depth bias (`SceneItem.DepthBias`, world units towards the camera, only the depth changes) and no longer flicker with the ground.
  - **Rings:** model formula corrected (child rings, not +0x71); one dial; hands only on rings that can be timed (`MapSceneBuilder.RingVariant`, `ModelVariant`, `RenderMesh.FromModel` with hidden and turned parts, cached per variant). Ring names in the list say "goal ring", "invisible check ring", "inactive at the start".
  - **Picking:** see-through surfaces (ring dials, hopper goal, Cannonball target) can be chosen wherever they are drawn (alpha > 0.02).

    - **View:** checkboxes for bilinear filtering, vertex shading, anti-aliasing (drawn at twice the size and averaged; off while dragging) and invisible objects; "Map info…" (`MapStatistics`: tiles, contours, triangles, vertices, decoration, textures and texels, objects, the level's files as stored, the measured `IslandBudget`).
    - **Camera:** buttons 3D (leave the axis view at the same place and visible size), Start view, Show all (from the current direction), Focus (on the chosen object). The axis views' focus is the middle of the picture. Orbit mode: choosing an object jumps to it, the "pan" button turns around it, zoom goes towards it. "Zoom" label at the zoom bar.
    - **Mouse modes:** only "Select and move" and "User defined"; the old "Navigate only" setting works like "Select and move".
    - **Handles:** world axes or object axes (`MapRenderSettings.GizmoRotation`); start arrows are longer and thicker (`SceneLine.Width`).
    - **Ever-Frost** with texture palette 5 (3.4); meshes are cached per palette.
    - **Sub-areas** in the island list (3.2); **island objects**: warp / area switch points (sphere, front arrow, line to the destination when chosen), sound zones, toy places, the objects placed by the game's code (3.3) with their circle when chosen (`MapScene.PathLines`). In the overview-map mode the list shows only the mission objects.
    - **Layout:** the bottom bar's sections fold away to the side, the bar scrolls and its height is set with a splitter; list : details = 1 : 2. **Credits** in the menu bar.
    - **Model names** from the user for 35 unnamed models (`ModelNames`); still unnamed: 0x03F, and the second "08f" of the user's list (rectangular flat building) needs its real number.
- Third rework of Step 51a (30 October 2026, user's test list):
    - **Mouse:** "Select and move": a left click chooses, holding the left button pans, the right button turns the view (orbit mode: around the object), the middle button pans. Orbit mode keeps the distance when another object is chosen in the list.
    - **Colours:** the renderer evaluates each texture's colour combiner (`Combiner`, `MapTexture.Detail`, paired images, primitive and environment colours, 3.5). Ever-Frost's land no longer shows the sea through cut-out texels, the Grand Canyon is brown. Opaque surfaces cut out texels only where the drawn alpha comes from the texture.
    - **Z-fighting:** decal materials are drawn 0.4 units towards the camera (depth only), as the game's decal mode.
    - **Overview map:** the pause map's markers instead of the mission objects (`SceneLayer.MapMarker`, 3.6); the list shows the objects with a marker. Rings, balloons and Floating Pads appear only in their area (island or cave).
    - **Shown now:** the planes and the ski lift on their spline paths (`SplinePath`, path when chosen), the fires (camera-facing flame picture), the sound emitters (point or line, near/far circles when chosen).

- Fourth rework of Step 51a (30 October 2026): separate checkboxes for vertex colours and lighting (`MapRenderSettings.VertexColors`, `Lighting`); a search box above the object list (name or group); the wheel zooms towards the point under the mouse (`MapCameraController.ZoomToward`; axis views keep that point under the mouse; orbit mode still zooms to the chosen object). Outside the map: the Text tab keeps its open groups and the chosen text when it is read again after saving or after list changes; PhotoScoringWindow's panel no longer hides `ContentControl.Content` (compiler warning).

9. Wishes of the user for later (29 October 2026)

- **Model tab:** export models, if possible as .obj and as a Google SketchUp file. Note: SketchUp's own format (.skp) can only be written with SketchUp's SDK (a C library under its own terms); SketchUp imports COLLADA (.dae), so a .dae export is the likely way. To be decided with the user when the model tab comes.
- **Credits** in the menu bar: done in the second rework of Step 51a.
- **Waves for own models:** the blue strips along the beaches are surf: triangles with textures that scroll (UVTX header: f32 scroll of the texture coordinates at 0x0C / 0x10 and of the combine at 0x04 / 0x08, e.g. Holiday's 0x1E, 0x28, 0x30, 0x03C, 0x03D) and fade out with the vertex alpha. Own models can get waves the same way: a band of triangles with such a texture. Remember for the model tab / model import (and possibly animate the scroll in the map view).
- Planes, ski lift, sound emitters and fires: shown since the third rework of Step 51a. Unused models of the game (e.g. 0x03F) come with adding objects.